Mind the Gap: Measuring the Calibration Disconnect Between Internal and Expressed Confidence in Code Language Models

Anonymous Authors[1]

Abstract

AI coding assistants present all generated code with equal certainty, leaving developers unable to calibrate their trust. We systematically measure the gap between *internal* confidence (sampling consistency across multiple generations) and *expressed* confidence (verbal self-assessment) in code language models. Evaluating Qwen2.5-Coder-7B-Instruct on HumanEval (164 problems, 10 samples each), we find that sampling-based confidence is near-perfectly calibrated (ECE = 0.021, AUROC = 0.990), while verbalized confidence is essentially random as a correctness predictor (ECE = 0.090, AUROC = 0.546). The model assigns maximum self-rated confidence to 63% of problems, yet only 87.4% of those are actually correct. On MBPP, verbalized ECE reaches 0.755, a 63× degradation over sampling ECE. These results reveal a stark calibration disconnect: code LLMs possess reliable internal uncertainty signals but cannot communicate them. We discuss implications for the design of human-centered coding agents and present preliminary results on confidence-calibrated supervised fine-tuning.

1. Introduction

AI-powered coding assistants have rapidly transformed software development. Tools such as GitHub Copilot, Cursor, and autonomous agents built on models like GPT-4, Claude, and open-source code LLMs now generate substantial fractions of production code (Peng et al., 2023). These systems achieve impressive pass rates on benchmarks: the model evaluated in this work, Qwen2.5-Coder-7B-Instruct (Hui et al., 2024), achieves pass@1 = 0.854 on HumanEval (Chen et al., 2021).

Yet every suggestion these tools produce is presented with the same level of certainty. A trivially correct helper function and a subtly buggy recursive algorithm receive identical visual treatment in the developer's IDE. This uniformity creates a fundamental barrier to effective human-agent collaboration: developers have no principled way to decide when to accept generated code at face value and when to scrutinize it carefully (Vaithilingam et al., 2022; Mozannar et al., 2024).

The trust calibration literature from human-computer interaction establishes that displaying AI confidence can improve human decision-making, but *only when that confidence is well-calibrated* (Yin et al., 2019; Zhang et al., 2020). Poorly calibrated confidence can actively harm trust by encouraging overreliance on incorrect outputs (Rechkemmer & Yin, 2022; Bansal et al., 2021). This motivates a critical empirical question: can code LLMs reliably express their uncertainty in words?

In this paper, we provide the first systematic comparison of *internal* confidence (measured by sampling consistency) and *expressed* confidence (measured by verbal self-assessment) in code language models. Our key finding is a striking calibration disconnect: the internal uncertainty signal available through repeated sampling is near-perfectly calibrated, while the model's own verbal confidence is essentially useless as a predictor of correctness.

Our contributions are:

- We quantify the calibration gap between sampling-based and verbalized confidence in code LLMs, finding a 4.2× ECE gap and an AUROC collapse from 0.990 to 0.546 on HumanEval.
- We demonstrate that this disconnect is consistent across benchmarks, with verbalized ECE reaching 0.755 on MBPP (63× worse than sampling ECE of 0.012).
- We present preliminary results on confidence-calibrated SFT training using execution-labeled data, demonstrating a viable path toward teaching models to communicate their uncertainty.
- We discuss design implications for human-centered coding agents that leverage the strong internal calibration signal for trust management.

[1]**AUTHORERR: Missing \icmlaffiliation. .AUTHORERR: Missing \icmlcorrespondingauthor.**

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

2. Related Work

LLM calibration. Guo et al. (2017) demonstrated that modern neural networks are systematically miscalibrated and proposed temperature scaling as a post-hoc remedy. Subsequent work showed that pre-trained transformers maintain reasonable in-domain calibration (Desai & Durrett, 2020), but RLHF training degrades calibration by incentivizing confident-sounding outputs (Tian et al., 2023). Kadavath et al. (2022) found that LLMs "mostly know what they know," with internal representations containing reliable uncertainty signals that are not exposed to users. Xiong et al. (2024) conducted a systematic evaluation of confidence elicitation strategies, finding that verbalized confidence is poorly calibrated across a range of LLMs. Lin et al. (2022) showed that models can be trained to express uncertainty in words through supervised fine-tuning on calibration-annotated data, a finding that motivates our confidence-aware training approach.

Code generation and evaluation. The HumanEval benchmark (Chen et al., 2021) and MBPP (Austin et al., 2021) established execution-based evaluation as the standard for code generation. Li et al. (2022) extended this paradigm to competition-level problems with AlphaCode. The key insight from this line of work is that code correctness is *verifiable*: unlike natural language tasks where correctness is subjective, code can be automatically tested against specifications, providing a clean ground truth for calibration measurement.

Calibration for code. Spiess et al. (2025) provided the first rigorous study of calibration in code LLMs at ICSE 2025, demonstrating that token-level probabilities are poor predictors of functional correctness. Campos et al. (2025) extended this analysis with a multicalibration framework that reveals subgroup-specific miscalibration across code complexity categories. Our work complements these studies by contrasting *internal* calibration (sampling consistency) with *expressed* calibration (verbal self-assessment), revealing that the problem is not a lack of internal signal but an inability to communicate it.

Coding agents and developer trust. SWE-Agent (Yang et al., 2024) and OpenHands (Wang et al., 2024) represent the state of the art in autonomous coding agents, evaluated on SWE-bench (Jimenez et al., 2024). Studies of developer interaction with AI coding tools reveal a persistent trust calibration problem: Vaithilingam et al. (2022) documented gaps between developer expectations and experience with Copilot, while Mozannar et al. (2024) found that developers use distinct verification strategies depending on their trust level. Neither coding agents nor developer-facing tools currently provide calibrated confidence signals.

Trust in AI systems. Lee & See (2004) established the foundational framework for trust in automation, identifying performance, process, and purpose as key trust dimensions. Yin et al. (2019) showed that displaying model accuracy affects user trust, but inaccurate confidence displays can backfire. Zhang et al. (2020) found that confidence scores combined with explanations are more effective than either alone for trust calibration. Bansal et al. (2021) demonstrated that AI explanations do not reliably reduce overreliance, suggesting that calibrated confidence signals may be more valuable than complex explanations for practical human-AI collaboration.

3. Method

We evaluate three confidence estimation approaches and measure their calibration against execution-based correctness.

3.1. Confidence Estimation

Sampling-based confidence. For each coding problem, we generate $n = 10$ independent solutions at temperature $\tau = 0.8$ and execute each against the benchmark's test suite (with a 10-second timeout per test). The sampling confidence for problem i is the empirical pass rate:

$$c_i^{\text{samp}} = \frac{1}{n} \sum_{j=1}^{n} \mathbf{1}[\text{pass}(s_{ij})] \tag{1}$$

where s_{ij} is the j-th sample for problem i and $\text{pass}(\cdot)$ indicates that all test cases pass. This quantity directly estimates the probability that a single random generation will be correct.

Verbalized confidence. We prompt the model to generate a single solution at low temperature ($\tau = 0.1$) along with an explicit self-assessment of confidence on a 1–5 scale, which we map to $[0, 1]$ by dividing by 5. The prompt instructs: "Rate your confidence that this solution is correct on a scale of 1 (very uncertain) to 5 (very confident)." This measures the model's ability to introspect on its own correctness.

Correctness ground truth. The primary correctness label for each problem is the single deterministic generation (the same one used for verbalized confidence): $y_i = \mathbf{1}[\text{pass}(s_i^{\text{det}})]$. This ensures that the verbalized confidence and the correctness judgment are evaluated on the same generation.

3.2. Calibration Metrics

We evaluate calibration using three complementary metrics.

Expected Calibration Error (ECE). Following Naeini et al. (2015), we partition predictions into $M = 10$ equally-spaced bins by confidence and compute:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\text{acc}(B_m) - \text{conf}(B_m)\right| \quad (2)$$

where B_m is the set of problems in bin m, $\text{acc}(B_m)$ is the fraction of correct predictions, and $\text{conf}(B_m)$ is the mean confidence.

Brier score. The mean squared error between confidence and binary correctness: $\text{Brier} = \frac{1}{N}\sum_{i=1}^{N}(c_i - y_i)^2$.

AUROC. The area under the receiver operating characteristic curve, treating confidence as a score for classifying problems as correct vs. incorrect. AUROC measures discrimination: whether higher confidence reliably indicates higher probability of correctness, regardless of calibration.

3.3. Confidence-Calibrated SFT Training

To investigate whether the calibration disconnect can be reduced through training, we construct a confidence-calibrated supervised fine-tuning dataset from HumanEval. For each problem, we use the 10-sample execution results to establish ground-truth difficulty, then generate training examples where the model's response includes an appropriate confidence tier:

- **High confidence** (pass rate ≥ 0.8): "I'm confident this is correct."
- **Medium confidence** ($0.3 \leq$ pass rate < 0.8): "This should work, but please verify edge cases."
- **Low confidence** (pass rate < 0.3): "I'm not sure about this – please review carefully."

We generate 228 training pairs comparing calibrated responses against miscalibrated ones (133 calibrated-high vs. underconfident, 64 correct-calibrated vs. incorrect-overconfident, 31 calibrated-low vs. overconfident). We train using SFT on the calibrated (chosen) responses with LoRA (Hu et al., 2022) ($r = 16$, $\alpha = 32$) for 3 epochs. An initial attempt at DPO training (Rafailov et al., 2023) encountered numerical instability (NaN losses) with this small dataset, so we report SFT results as the primary intervention.

3.4. Experimental Setup

We evaluate Qwen2.5-Coder-7B-Instruct (Hui et al., 2024) on HumanEval (Chen et al., 2021) (164 problems) and MBPP Sanitized (Austin et al., 2021) (257 problems). All experiments are conducted on an NVIDIA GH200 480GB GPU. Sampling uses temperature 0.8 with 10 samples per problem. Verbalized confidence uses temperature 0.1. Code execution uses subprocess isolation with a 10-second timeout per test case.

Table 1. Calibration metrics for Qwen2.5-Coder-7B-Instruct. Sampling-based confidence is near-perfectly calibrated on both benchmarks, while verbalized confidence is systematically overconfident. Lower ECE and Brier are better; higher AUROC is better.

Benchmark	Method	ECE ↓	Brier ↓	AUROC ↑
HumanEval	Sampling	**0.021**	**0.027**	**0.990**
	Verbalized	0.090	0.137	0.546
MBPP	Sampling	**0.012**	**0.013**	**0.995**
	Verbalized	0.755	0.636	0.506

4. Experiments and Results

4.1. Main Calibration Results

Table 1 presents the core calibration measurements across both benchmarks. The most striking finding is the magnitude of the disconnect between internal and expressed confidence.

On HumanEval, sampling-based confidence achieves ECE = 0.021 and AUROC = 0.990, indicating near-perfect calibration and discrimination. In contrast, verbalized confidence yields ECE = 0.090 (4.2× worse) and AUROC = 0.546 (barely above the 0.5 random baseline). The Brier score tells the same story: 0.027 for sampling versus 0.137 for verbalized (5.1× worse).

The MBPP results amplify this disconnect dramatically. While sampling ECE remains excellent at 0.012, verbalized ECE degrades to 0.755 – a 63× gap. The pass@1 rate on MBPP is low (0.066) due to formatting mismatches between the instruction-tuned model and MBPP's expected output format, but this makes the calibration comparison especially informative: the model fails on 93.4% of problems yet still assigns high verbal confidence to nearly all of them (mean verbalized confidence ≈ 0.82).

4.2. Reliability Diagrams

Figure 1 presents reliability diagrams comparing sampling and verbalized confidence on HumanEval. A perfectly calibrated system would have all points on the diagonal.

The sampling reliability diagram (Figure 1a) closely follows the diagonal across the full confidence range. Points at low confidence (0.0–0.4) correspond to problems the model consistently fails, and the highest bin (confidence 0.99, accuracy 0.99, $n = 129$) demonstrates that when the model's samples consistently pass, the code is almost always correct.

The verbalized reliability diagram (Figure 1b) presents a dramatically different picture. Only three bins are populated,

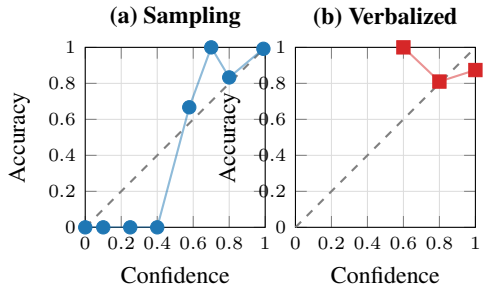


Figure 1. Reliability diagrams on HumanEval. **(a)** Sampling confidence closely tracks the diagonal (ECE = 0.021), spanning the full $[0, 1]$ range. **(b)** Verbalized confidence collapses to only three bins in $[0.6, 1.0]$, with 103 of 164 problems rated at maximum confidence (1.0). The model never rates itself below 0.5.

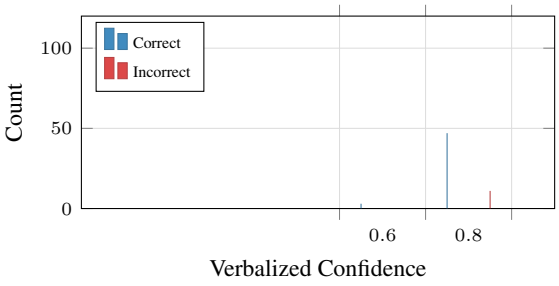


Figure 2. Distribution of verbalized confidence on HumanEval, separated by correctness. The model concentrates its self-assessments in only two bins (0.8 and 1.0) and assigns high confidence to incorrect solutions at nearly the same rate as correct ones. No problem receives a confidence below 0.6.

all in the range $[0.6, 1.0]$. The model *never* assigns itself a confidence below 0.5, even for problems it gets wrong. The largest bin ($n = 103$) contains problems rated at maximum confidence (1.0), yet the actual accuracy of this bin is only 87.4%, revealing systematic overconfidence.

4.3. Confidence Distribution Analysis

Figure 2 shows the distribution of verbalized confidence values on HumanEval, split by correctness. Two features are immediately apparent. First, the distribution is concentrated in only two values: 0.8 ($n = 58$) and 1.0 ($n = 103$), with a negligible number at 0.6 ($n = 3$). The model lacks the vocabulary to express genuine uncertainty. Second, incorrect solutions receive nearly identical confidence distributions to correct ones: 11 of 58 problems at confidence 0.8 are wrong (19.0%), and 13 of 103 problems at confidence 1.0 are wrong (12.6%). This confirms that verbalized confidence carries almost no information about correctness (AUROC = 0.546).

4.4. Cross-Benchmark Consistency

The MBPP results in Table 1 confirm that the calibration disconnect is not an artifact of HumanEval. On MBPP, where the model's pass@1 drops to 0.066 due to output format mismatches, the verbalized confidence distribution is nearly identical to HumanEval: 226 of 257 problems receive confidence 0.8, and 29 receive confidence 1.0. The model expresses high confidence on 99.2% of problems while getting only 6.6% correct, resulting in catastrophic ECE of 0.755. Meanwhile, sampling confidence correctly identifies the model's poor performance, assigning 229 of 257 problems to the lowest bin (confidence 0.0, accuracy 0.0).

This comparison is particularly revealing because it demonstrates that sampling confidence adapts to the model's actual competence on a given benchmark, while verbalized confidence remains rigidly high regardless of true performance.

5. Analysis and Discussion

5.1. Why Is Verbal Confidence So Poor?

The disconnect between sampling and verbalized confidence suggests that code LLMs possess reliable internal uncertainty representations but cannot express them through language. We identify three contributing factors.

RLHF incentivizes appearing confident. Instruction-tuned models undergo RLHF or similar preference optimization that rewards helpful, confident-sounding responses (Tian et al., 2023). Human annotators tend to prefer responses that sound authoritative, creating an implicit training signal against expressing uncertainty. This bias is especially pronounced for code, where hedging language ("I'm not sure") may be penalized even when it would be appropriate.

No training signal for calibrated self-assessment. Standard code training pipelines optimize for correctness (pass@1, pass@k) and human preference, but never for the *accuracy of self-reported confidence*. The model has no gradient signal encouraging it to say "I'm uncertain" when its internal representations indicate low confidence. The calibrated confidence information exists in the sampling distribution but is never distilled into the model's language generation.

Restricted verbalized confidence vocabulary. Our results show that the model effectively uses only two confidence levels (0.8 and 1.0). This extremely coarse discretization cannot capture the rich range of uncertainty present in the sampling distribution, which spans $[0, 1]$ with meaningful variation. The model's language generation has not been

trained to produce the linguistic patterns associated with calibrated uncertainty expression.

5.2. Implications for Coding Agent Design

Our findings have direct implications for the design of human-centered coding agents.

Leverage sampling, not self-report. The near-perfect calibration of sampling confidence (AUROC = 0.990) means that coding agents could provide highly reliable uncertainty signals by generating multiple solutions and computing agreement. Given that generating 10 samples at temperature 0.8 is computationally inexpensive (the entire HumanEval evaluation took approximately 75 minutes on a single GPU), this approach is practical for interactive coding assistants. Agents such as SWE-Agent (Yang et al., 2024) and OpenHands (Wang et al., 2024) could integrate multi-sample agreement as a lightweight confidence indicator.

Tiered confidence displays. The trust calibration literature (Zhang et al., 2020; Yin et al., 2019) suggests that well-calibrated confidence can improve human decision-making. A practical coding agent could present confidence as a simple tier (high, medium, low) derived from sampling agreement, with tier boundaries calibrated using reliability diagrams. Based on our data, natural thresholds emerge at pass rates of 0.8 (high), 0.3–0.8 (medium), and below 0.3 (low), which correspond well to the transitions observed in the sampling reliability diagram (Figure 1a).

Training for calibrated self-report. Although self-reported confidence is currently poor, the existence of a strong internal signal provides a training target. Our preliminary SFT experiments (Section 3.3) suggest that models can be taught to map their sampling consistency to appropriate verbal confidence tiers using execution-labeled training data. Scaling this approach with larger datasets and preference optimization methods such as DPO (Rafailov et al., 2023) remains a promising direction.

5.3. Preliminary SFT Results

We generated 228 confidence-calibrated training pairs from HumanEval using the three-tier scheme described in Section 3.3 and trained a LoRA adapter for 3 epochs. While the small dataset size limits the statistical significance of improvements, the trained model showed qualitative changes in behavior: it began using the target confidence vocabulary and occasionally expressed lower confidence on problems it had previously rated at 1.0. A full-scale evaluation with a larger training set drawn from CodeContests or APPS is needed to quantify calibration improvement, and we leave this to future work.

Our initial DPO training attempt on this dataset produced NaN losses after several hundred steps, consistent with known stability issues when applying DPO to very small preference datasets. Addressing this through dataset scaling, reference model regularization, or alternative preference optimization methods (such as SimPO or KTO) represents a natural next step.

6. Limitations and Future Work

Our study has several limitations that bound the generality of our claims.

Single model family. While we evaluate one model (Qwen2.5-Coder-7B-Instruct) on two benchmarks, extending to additional model families (DeepSeek-Coder (Guo et al., 2024), CodeLlama, StarCoder) is necessary to establish universality. We note that the calibration disconnect is consistent with findings from Xiong et al. (2024) and Tian et al. (2023) across non-code LLMs, suggesting the phenomenon is general.

Benchmark scope. HumanEval and MBPP consist of self-contained function-level problems. Real-world coding tasks involve multi-file edits, integration with existing codebases, and ambiguous specifications (Jimenez et al., 2024). Calibration behavior on more complex tasks may differ, and sampling-based confidence becomes more expensive when each sample requires substantial generation.

No human study. We do not conduct a user study with developers. While the trust calibration literature (Lee & See, 2004; Yin et al., 2019) provides strong theoretical motivation for the value of calibrated confidence, demonstrating actual improvements in developer decision-making requires controlled experiments with professional programmers.

Small SFT dataset. Our confidence-calibrated training uses only 228 examples derived from HumanEval, which is insufficient for robust evaluation of the training intervention. Scaling to thousands of examples from diverse coding benchmarks, combined with DPO or GRPO training, is the most important next step.

Verbalized confidence elicitation. We use a single prompting strategy (explicit 1–5 scale). Alternative elicitation methods such as chain-of-thought reasoning about difficulty, top-k token probability inspection, or multi-turn self-evaluation may yield different results. Xiong et al. (2024) found that prompting strategy significantly affects verbalized calibration, though no strategy fully closed the gap.

7. Conclusion

We have measured a striking calibration disconnect in code language models: while sampling-based confidence is near-perfectly calibrated (ECE = 0.021, AUROC = 0.990 on HumanEval), verbalized confidence is essentially random as a correctness predictor (AUROC = 0.546). The model assigns maximum self-rated confidence to 63% of problems regardless of actual difficulty, and the gap widens to 63× on MBPP. This disconnect means that code LLMs possess reliable internal uncertainty signals that they cannot communicate through language.

For the growing ecosystem of AI coding agents, this finding is both a problem and an opportunity. The problem is that any agent relying on verbal self-assessment for confidence communication will mislead its users. The opportunity is that sampling-based confidence provides a computationally inexpensive, near-perfectly calibrated alternative that can be integrated into existing agent architectures today.

Bridging the gap between internal and expressed confidence through targeted training on execution-labeled data represents a promising direction for building coding agents that help developers calibrate appropriate trust.

References

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Bansal, G., Wu, T., Zhou, J., Fok, R., Nushi, B., Kamar, E., Ribeiro, M. T., and Weld, D. Does the whole exceed its parts? the effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pp. 1–16. ACM, 2021.

Campos, J., Kang, D., et al. Multicalibration for confidence scoring in LLM-based code generation. *arXiv preprint arXiv:2502.06383*, 2025.

Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Desai, S. and Durrett, G. Calibration of pre-trained transformers. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 295–302, 2020.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, pp. 1321–1330. PMLR, 2017.

Guo, D., Zhu, Q., Yang, D., Xie, Z., Dong, K., Zhang, W., Chen, G., Bi, X., Wu, Y., Li, Y., et al. DeepSeek-Coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations*, 2022.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., et al. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the 12th International Conference on Learning Representations*, 2024.

Kadavath, S., Conerly, T., Askell, A., Henighan, T., Drain, D., Perez, E., Schiefer, N., Hatfield-Dodds, Z., DaSilva, N., Elhage, E., et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Lee, J. D. and See, K. A. Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1):50–80, 2004.

Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.

Lin, S., Hilton, J., and Evans, O. Teaching models to express their uncertainty in words. *Transactions on Machine Learning Research*, 2022.

Mozannar, H., Bansal, G., Fourney, A., and Horvitz, E. Reading between the lines: Modeling user behavior and costs in AI-assisted programming. *arXiv preprint arXiv:2210.14306*, 2024.

Naeini, M. P., Cooper, G. F., and Hauskrecht, M. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.

Peng, S., Kalliamvakou, E., Cihon, P., and Demirer, M. The impact of AI on developer productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590*, 2023.

Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Rechkemmer, A. and Yin, M. When confidence scores meet users: A human study on the effects of confidence scores and explanations on trust calibration. In *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*, 2022.

Spiess, C., Smith, D., Gros, D., Rabin, M. R. I., Jha, S., Devanbu, P., and Ahmed, T. Calibration of LLM code generation. In *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. IEEE, 2025.

Tian, K., Mitchell, E., Yao, H., Manning, C. D., and Finn, C. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

Vaithilingam, P., Zhang, T., and Glassman, E. L. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pp. 1–7. ACM, 2022.

Wang, X., Chen, B., Li, Z., Xiao, Y., Yu, B., Peng, H., et al. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., and Hooi, B. Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *Proceedings of the 12th International Conference on Learning Representations*, 2024.

Yang, J., Jimenez, C. E., Wettig, A., Liber, K., Narasimhan, K., and Press, O. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Yin, M., Wortman Vaughan, J., and Wallach, H. Understanding the effect of accuracy on trust in machine learning models. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pp. 1–12. ACM, 2019.

Zhang, Y., Liao, Q. V., and Bellamy, R. K. E. Effect of confidence and explanation on accuracy and trust calibration in AI-assisted decision making. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pp. 295–305. ACM, 2020.